COMPUTER VISION

Exercise 1

Digital Images and Basic Image Processing

Learning objectives

After completing this exercise, you should be able to:

- Read, display, and inspect a grayscale image using OpenCV and NumPy.
- Explain image dimensions, pixel coordinates, data types, and intensity values.
- Apply basic pixel-wise transformations and handle valid intensity ranges.
- Plot a histogram and perform a simple contrast-stretching operation.
- Understand nearest-neighbor zooming and a basic image translation.
- Compare pixel replication and zero-order hold with other image-zooming methods.
- Apply logarithmic and gamma correction and implement histogram equalization.
- Perform geometric rotation and explore additional intensity transformations.

Instructions

- Use Python 3 with NumPy, Matplotlib, and OpenCV (cv2).
- Submit one Jupyter Notebook (.ipynb) containing code, outputs, and short written answers.
- Add a title to every figure and label each subplot so that the original and processed images are easy to identify.
- For parts that explicitly ask you to implement an operation, do not use a library function that performs that exact operation. You may use OpenCV afterward to check your result.
- Use the supplied image files from the companion assets folder. Keep the files beside your notebook or update the paths in your code.
- Answer every sub-question. Add notebook cells wherever needed to keep code, outputs, and explanations clear.

Learning note: Most supplied images are 8-bit grayscale images. Their pixel intensities are integers from 0 (black) to 255 (white). For a grayscale NumPy image, use image[row, column] to access a pixel.

Question 1 — Working with a Digital Image (15 points)

Use camera.png for this question. It is a 512 × 512 grayscale image.



Figure 1. camera.png — input image for Questions 1, 2, and 4. Source: scikit-image sample data (CC0).

A) Read camera.png in grayscale using cv2.imread and display it using Matplotlib. (3 points)

B) Print the image shape, data type, minimum pixel value, and maximum pixel value. Briefly explain what each result tells you. (4 points)

C) Report the intensity value at image[100, 150]. Then crop a 30 × 30 region starting at row 100, column 150, and display the crop. (4 points)

D) Explain why a grayscale image is usually represented by a 2D array, while a color image is commonly represented by a 3D array. In your explanation, distinguish row/column indexing from x/y coordinates. (4 points)

Rows (y) \ Columns (x)	0	1	2	3	4	5	6	7
0	0	32	64	96	128	160	192	224
1	16	48	80	112	144	176	208	240
2	24	56	88	120	152	184	216	248
3	8	40	72	104	136	168	200	232

Figure 2. A small intensity matrix: the first index denotes the row; the second denotes the column.

Learning note: In NumPy, image[row, column] is the standard indexing order. In geometric descriptions, x usually refers to the column (horizontal position) and y to the row (vertical position).

Question 2 — Basic Pixel-wise Operations (20 points)

Use camera.png again. Create three new images by changing pixel intensities.

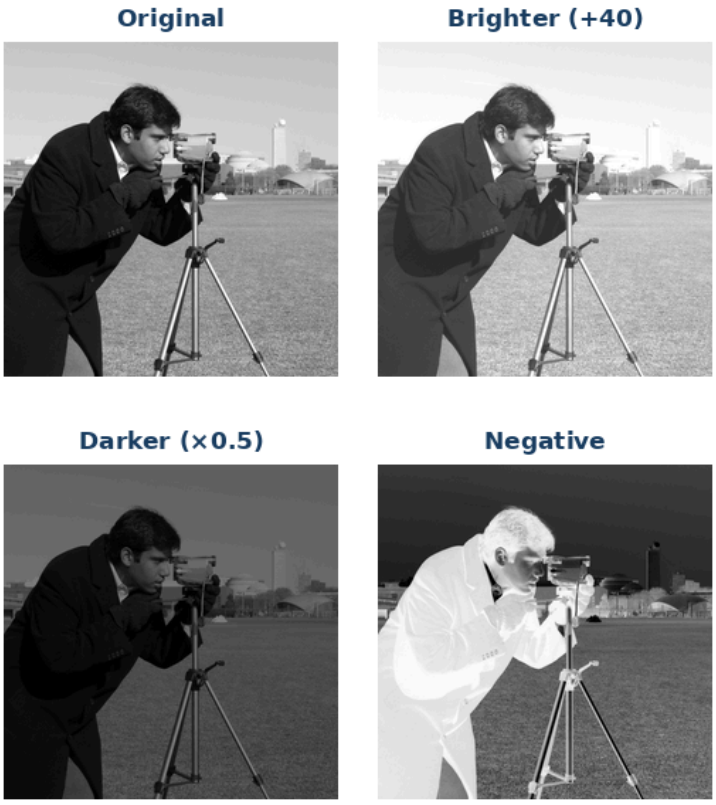


Figure 3. Qualitative guide to the expected appearance of the transformations; you must implement them yourself.

A) Increase every pixel intensity by 40. Ensure that the output values remain in the valid 0–255 range. (5 points)

B) Create a darker version by multiplying every pixel intensity by 0.5. Convert the result back to an 8-bit image. (4 points)

C) Create the negative image using the formula below. (4 points)

$$I_{negative} = 255 - I$$

D) Display the original image and all three outputs in a 2 × 2 grid. Give each subplot a descriptive title and write two or three sentences comparing the results. (4 points)

E) A pixel has intensity 230. If 40 is added, what value should be stored in the output image? Explain why the raw arithmetic result cannot be kept in an 8-bit grayscale image. (3 points)

Learning note: An 8-bit grayscale image stores values from 0 to 255. A safe approach is to perform arithmetic in a wider numeric type, clip values to [0, 255], and then convert to uint8. Direct arithmetic on uint8 arrays may produce unexpected wrap-around behavior.

Question 3 — Histogram and Contrast Stretching (25 points)

Use low_contrast.png from the companion assets. This image was prepared by compressing the intensity range of camera.png so that its contrast is reduced.



Figure 4. low_contrast.png — prepared input for contrast stretching.

A) Read and display the image. Plot its grayscale histogram. Label the horizontal axis “Pixel intensity” and the vertical axis “Number of pixels”. (6 points)

B) Find the minimum and maximum pixel values. From the histogram, describe where most intensity values are concentrated and explain what this suggests about the image contrast. (4 points)

C) Implement linear contrast stretching using the formula below. Use the minimum and maximum values of the input image. Your code must handle the special case in which the minimum equals the maximum. (8 points)

$$I_{stretched} = 255 \times (I - I_{min}) / (I_{max} - I_{min})$$

D) Display the original and stretched images side by side and plot both histograms in the same figure or in clearly labeled subplots. (4 points)

E) Explain whether contrast stretching can always recover information that was not visible in the original image. What should your program do if all pixels have the same intensity? (3 points)

Learning note: Contrast stretching expands the range of intensity values and may improve visibility. It does not automatically reconstruct information that is absent from the input. If I_max = I_min, the formula has a zero denominator; handle this case explicitly.

Question 4 — Image Zooming (20 points)

Use camera.png. Select a 64 × 64 crop from a region that contains visible edges or texture. Perform 2× zooming with nearest-neighbor pixel replication.

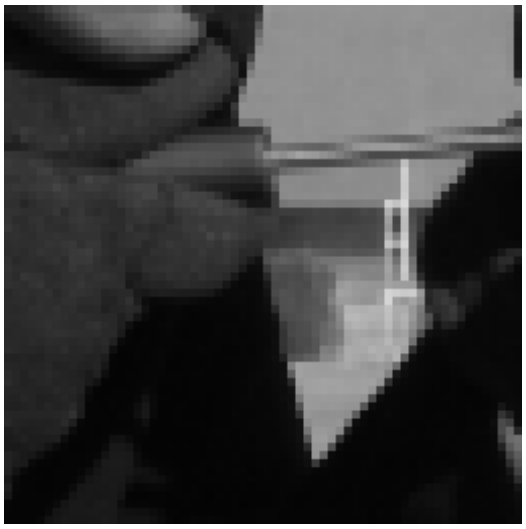


Figure 5. Example of nearest-neighbor enlargement. This illustration is for visual reference; use a crop from camera.png in your implementation.

A) In your own words, describe how nearest-neighbor zooming determines the values of new pixels. (4 points)

B) Implement 2× zooming on your crop using loops and array indexing. Do not use cv2.resize or np.repeat in this part. Each source pixel should fill a 2 × 2 block in the output. (8 points)

C) Resize the same crop using cv2.resize with interpolation=cv2.INTER_NEAREST. Compare your result with the implementation from Part B. Are the output dimensions and pixel values the same? (4 points)

D) Repeat the zoom with a factor of 4. Briefly describe how the appearance of edges and fine details changes as the zoom factor increases. (4 points)

Supplementary zooming task from the second version (additional marks, if assigned separately)

Use zoom_target.png to compare Pixel Replication and Zero-order Hold. These tasks extend the core nearest-neighbor exercise above.

Figure 7. zoom_target.png - reference input for the supplementary zooming task.



E) Briefly explain how Pixel Replication and Zero-order Hold assign values to pixels in an enlarged image.

F) Implement both methods in Python and enlarge the image by a factor of 3. Do not use a built-in resizing function in either implementation.

G) Display the original image and both enlarged results in one figure. Print the height and width of each image.

H) Repeat both methods with a factor of 5. Show a close-up region from the factor-3 and factor-5 results and compare the visible details.

I) Name one other image-zooming method and briefly compare its results with Pixel Replication and Zero-order Hold.

Learning note: For a zoom factor of 2, the output dimensions are twice the input height and width. Nearest-neighbor zooming copies existing values; it does not create new image detail.

Question 5 — A Basic Geometric Translation (20 points)

Use square.png. The background intensity is 0, the small square has intensity 150, and the large square has intensity 220. The small square occupies rows 40:80 and columns 40:80. The large square must remain unchanged.

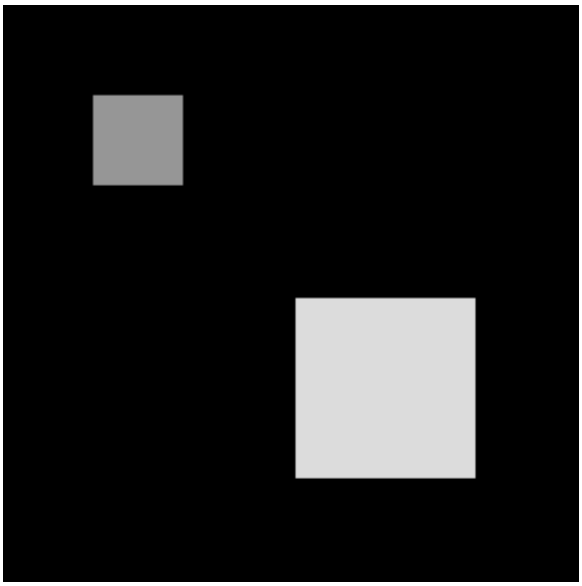


Figure 6. square.png — synthetic image for practicing pixel coordinates and translation.

A) A translation moves an object without changing its size. If the small square is moved 30 pixels to the right and 20 pixels down, what are the changes to its column and row indices? (5 points)

B) Create a translated image in which only the small square (intensity 150) moves 30 pixels right and 20 pixels down. Keep the large square (intensity 220) fixed. Use array indexing/slicing; do not use cv2.warpAffine for this part. (8 points)

C) Display the original and resulting images side by side. Check that the number of pixels with intensity 150 is unchanged before and after the translation. (4 points)

D) Explain why a translated object may be partly lost if it moves beyond the image boundaries. (3 points)

Learning note: Geometric transforms change where pixel values appear. Unlike Question 2, a translation is not intended to change the intensity value of each object pixel. In NumPy, rows increase downward and columns increase to the right.

Supplementary Questions from the Second Version

Questions 1-5 above retain the original 100-point score. The following added topics are part of the combined exercise; the instructor may assign marks for them separately.

Question 6 - Logarithmic and Gamma Correction

Use low_contrast_scene.png, a grayscale image with a narrow range of intensity values, and apply logarithmic and gamma correction.

Figure 8. low_contrast_scene.png - input for logarithmic and gamma correction.



A) Read the image and normalize its pixel values to the range [0, 1].

B) Briefly describe the purpose of logarithmic correction and gamma correction.

C) Apply the logarithmic transformation g(x,y) = c log(1 + f(x,y)). Choose c so the result is scaled to [0, 1].

D) Apply gamma correction twice, using gamma = 0.5 and gamma = 1.5. Keep the output values in the range [0, 1].

E) Display the original and all transformed images. Plot the histograms of the original, logarithmic, and two gamma-corrected images on one figure. Label the axes with intensity (0 to 1) and frequency.

Question 7 - Histogram Equalization

Use equalization_scene.png to perform histogram equalization and compare the image before and after processing.

Figure 9. equalization_scene.png - input for histogram equalization.



A) Read the image and normalize its pixel values to the range [0, 1].

B) Calculate and plot the histogram of the original image.

C) Implement histogram equalization using the cumulative histogram, then calculate and plot the histogram of the result.

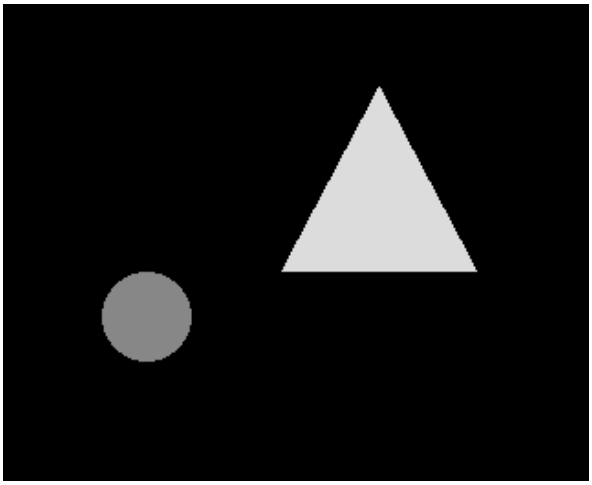
D) Present the original image, the equalized image, and both histograms together in one figure.

E) Compare the original and equalized images. Describe how the distribution of pixel intensities changed after equalization.

Question 8 - Geometric Transformations: Translation and Rotation

The image geometric_shapes.png contains a small circle with intensity 135 and a large triangle with intensity 220 on a black background. Read the image and perform both transformations below.

Figure 10. geometric_shapes.png - input for the geometric transformations task.



A) Translate the small circle 35 pixels to the right and 25 pixels downward.

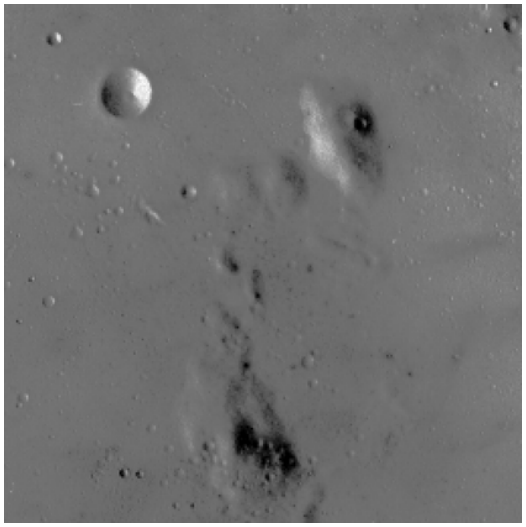
B) Rotate the large triangle 45 degrees counter-clockwise.

C) Write Python code to perform both transformations. Display the original image and the transformed result side by side.

Question 9 - Additional Intensity Transformations

For intensity_transform_scene.png, normalize the image to [0, 1] and apply each of the following transformations separately.

Figure 11. intensity_transform_scene.png - input for the additional intensity transformations.



1) Exponential transformation: g(x,y) = exp(c f(x,y)). Choose a suitable value of c and, if needed for display, rescale the result to [0, 1]. State what you did.

2) Logarithmic transformation: g(x,y) = c log10(f(x,y) + 1). Choose c so the output is in [0, 1].

3) Piecewise transformation: g(x,y) = f(x,y) when f(x,y) < 0.1; otherwise g(x,y) = 2 f(x,y). Keep the displayed output in a valid range.

A) Generate one output image for each of the three transformations.

B) Display the original image and the three output images together in one figure.

C) Which transformation gives the most useful visual result for this image? Briefly explain your choice.

Submission checklist

- Submit one .ipynb notebook with all cells executed and visible outputs.
- Include the required input assets: camera.png, low_contrast.png, square.png, zoom_target.png, low_contrast_scene.png, equalization_scene.png, geometric_shapes.png, and intensity_transform_scene.png (or submit the provided asset bundle unchanged).
- Give every figure a title and label axes on the histogram plots.
- Write brief explanations in your own words; screenshots without code or interpretation are incomplete answers.